USER STORIES

1. **Start A Game**

Actor *#who or what triggers this user story?*

- Player (clicks start game)

Preconditions *#what must be true before this use case can run?*

- Game app is already launched

Main Flow *#step by step sequence of how this user story runs*

- Player clicks "Start Game"
- System prompts for number of players
- Player enters a number between 2 and 5 (n)
- System creates Player objects (player1, player2, etc.)
- System creates a full Deck object and populates it with all card info.
- System removes all Exploding Kitten cards and Defuse cards from the deck
- System shuffles the clean deck
- System deals 7 cards to each Player's hand array
- System gives each Player 1 Defuse card (added directly to their hand)
- System reintroduces (n-1) Exploding Kitten cards back into the deck
- System returns remaining Defuse cards back into the deck
- System shuffles the deck again
- System initializes a GameState object with:
    - the deck,
    - an empty discard pile,
    - all players,
    - current_player_index to 0 # current player playing
    - Turns_to_play = 1 # current player should play one turn
- Game begins — first player's turn starts

Alternate Flows *#anything that can deviate from the normal sequence*

User enters invalid number of players

- System shows an error screen
- Resumes at step 2.

Postconditions *#what is guaranteed to be true after this use case runs.*

- Each Player object has exactly 8 cards in their hand (7 random + 1 Defuse)
- The Deck contains exactly (n-1) Exploding Kitten cards plus all other cards (shuffled)
- A GameState object exists and is active
- The discard pile is empty
- current_player_index is set to 0 (or randomized if you want a random first player)
- All players have is_eliminated = False

NB: When an exploding kitten card is defused, it’s then added back into the deck to ensure that the total number of exploding kitten cards in the deck at each point == n -1.

2. **<u>Play A Turn</u>**

<u>Actor</u> *#who or what triggers this user story?*

- Player whose turn it is

<u>Preconditions</u> *#what must be true before this use case can run?*

- A game is active (GameState exists and status is "active")
- It is this player's turn (current_player_index points to this player)
- The player is not eliminated (is_eliminated == False)
- The deck has at least one card in it

<u>Main Flow</u> *#step by step sequence of how this user story runs*

1. System highlights the current player's turn (UI indicates whose turn it is)
2. Player presses play a turn button
3. System displays two buttons: "**play a card"** and "**done playing cards"**
4. Player chooses to play a card
5. Player selects a card(s) (can be a combo) to play from their hand if any
6. System sets game state to be "pending_action"
7. The system checks if the card can be played or if that combo can be played. → returns True or False
8. System clears the "pending action" state from game state
9. If can be played (i.e no nopes and can be executed)
    a. System executes the card
10. System moves the played card(s) to the DiscardPile
11. System removes the card(s) from the Player's hand
12. Return to step 3
13. Player chooses "**done playing"**
14. System checks if skip_draw is True
15. If no (not skipping):
    a. Player clicks "Draw a Card"
    b. System pops the top card off the Deck
    c. System checks if the drawn card is an Exploding Kitten
        i. If yes:
            1. trigger Use Case Remove Player (which internally checks for Defuse)
        ii. If no:
            1. System adds the card to the Player's hand
16. System resets skip_draw to False
17. System checks if "is_attacking" (just played an attack card)
    a. Is_attacking is set back to False (for the next person)
    b. skip to step 20.
18. If "turns_to_take" > 0:(still have more turns to take i.e if you are being attacked)
    a. Decrement "turns to take"
    b. Check again if turns to take == 0: Go to step 3 again # *i.e under Attack*
19. System explicitly resets turns to take == 0 (should be 0 though)
20. System advances current_player_index to the next active player

<u>Alternate Flows</u> *#anything that can deviate from the normal sequence*

- In case the players hand is empty and they have no cards
    - Maybe have sth like if no cards are in the hand just show the **"done playing"** button

    - Can the deck ever be empty?
    - What if a player who is not currently playing attempts to play?

<u>Postconditions</u> *#what is guaranteed to be true after this use case runs.*

- System is set for another person to play
    - Current_player_index points to the new layer
    - Turns_to_take is updated
    - Skip_draw is updated

3. **<u>Remove Player</u>**

<u>Actor</u> *#who or what triggers this user story?*

- System triggers this when it notices a player selected an “exploding kitten card”

<u>Preconditions</u> *#what must be true before this use case can run?*

- The game is active (GameState status == "active")
- It is this player's turn
- The player just drew an Exploding Kitten card from the top of the Deck
- The player is not already eliminated (is_eliminated == False)

<u>Main Flow</u> *#step by step sequence of how this user story runs*

1. System checks if player has a defuse card present
2. If they have it:
    a. Call the “Defuse” User story
3. If they don’t:
    a. System updates Player status in the game state to is_eliminated = True.
    b. System puts all cards in the eliminated Player's hand to the Discard Pile
    c. System removes all cards from the player's hand leaving it empty. == []
    d. System adds that exploding kitten card to the discard pile
    e. System removes the Player from the active turn rotation.
    f. System shows player a screen of a dying kitten
    g. Player dismisses the screen.
    h. System checks the count of players where is_eliminated == False if it is equal to 1(number of alive players).
    i. If yes:
        i. System triggers Use Case “EndGame” if the count equals 1.
    j. If no: # only that player is being removed
        i. Go back to PlayATurn step 20.

<u>Alternate Flows</u> *#anything that can deviate from the normal sequence*

<u>Postconditions</u> *#what is guaranteed to be true after this use case runs.*

- The Exploding Kitten card is either back in the Deck (if Defused) or in the DiscardPile (if not)
- If eliminated: player.is_eliminated == True,
    - player.hand == []
    - GameState accurately reflects one fewer active player
    - The DiscardPile contains all of the eliminated player's former cards (if eliminated)
- Else:
    - Turns to take is back to 1
    - current_player_index now points to the next active player
    - System ready for the next PlayATurn (all play a turn preconditions are satisfied)

4. **Defuse**

Actor *#who or what triggers this user story?*

- System triggers this case when a player has drawn an exploding kitten

Preconditions *#what must be true before this use case can run?*

- It is this players turn
- Player just drew an exploding kitten

Main Flow *#step by step sequence of how this user story runs*

- System asks the player where they want to insert the exploding kitten card
- Player inserts a position (number) within the length of the deck
- System inserts exploding kitten card in that slot
- System removes the defuse card from the players hand
- System adds the defuse to the discard pile

Alternate Flows *#anything that can deviate from the normal sequence*

Postconditions *#what is guaranteed to be true after this use case runs.*

- Exploding Kitten card is back in the deck
- Player's is_eliminated state = False (player is left still alive)
- Moves back to PlayATurn step 18 (system decrements turns_to_take)

5. **<u>End Game</u>**

<u>Actor</u> *#who or what triggers this user story?*

- System when only one player is_eliminated == False

<u>Preconditions</u> *#what must be true before this use case can run?*

- Only one player is_eliminated == False (only one guy is alive)
- The game is active (GameState status == "active")
- Remove Player use case has just completed

<u>Main Flow</u> *#step by step sequence of how this user story runs*

- System sets game state to “ended”
- System displays winner on screen
- System shows two buttons “ResetGame” or “Close”
- Reset Game
    - System resets everything (preconditions for “Start A Game”)
    - Calls user story Start A Game from the top.
- Close
    - Shuts down game application
    - Maybe goes back to landing screen

<u>Alternate Flows</u> *#anything that can deviate from the normal sequence*

- What to do when a user just closes the tab?
    - Just end the game

<u>Postconditions</u> *#what is guaranteed to be true after this use case runs.*

- Either the game is shut down or a new game is reset.

6. **<u>Shuffle</u>**

<u>Actor</u> *#who or what triggers this user story?*

- Player selected this card

<u>Preconditions</u> *#what must be true before this use case can run?*

- It is this player's turn

<u>Main Flow</u> *#step by step sequence of how this user story runs*

- System shuffles the deck

<u>Alternate Flows</u> *#anything that can deviate from the normal sequence*

- Can deck be empty? → just stays empty?
- Nope case? → already handled before this is called in the pending action step

<u>Postconditions</u> *#what is guaranteed to be true after this use case runs.*

- Deck is shuffled
- Moves back to PlayATurn step 10.

7. **<u>Skip</u>**

<u>Actor</u> *#who or what triggers this user story?*

- Player selected this card

<u>Preconditions</u> *#what must be true before this use case can run?*

- It is this player's turn

<u>Main Flow</u> *#step by step sequence of how this user story runs*

- System sets skip_draw = True

<u>Alternate Flows</u> *#anything that can deviate from the normal sequence*

<u>Postconditions</u> *#what is guaranteed to be true after this use case runs.*

- Moves back to PlayATurn step 10.

8. **<u>See The Future</u>**

<u>Actor</u> *#who or what triggers this user story?*

- Player selected this card

<u>Preconditions</u> *#what must be true before this use case can run?*

- It is this players turn

<u>Main Flow</u> *#step by step sequence of how this user story runs*

- System reads the top 3 card names
- System shows the player the card names
    - # we were thinking add the cards to the player's object instance to prevent other players from seeing the cards.
- Player clicks "Im Done Seeing"
- System clears the future cards from the players object

<u>Alternate Flows</u> *#anything that can deviate from the normal sequence*

- Deck has less than 3 cards (show all cards <= 3)

<u>Postconditions</u> *#what is guaranteed to be true after this use case runs.*

- Deck is unchanged
- Future_cards field in the player's object is now empty
- Moves back to PlayATurn step 10.

9. **<u>Favor</u>**

<u>Actor</u> *#who or what triggers this user story?*

- Player selected this card

<u>Preconditions</u> *#what must be true before this use case can run?*

- It is this players turn

<u>Main Flow</u> *#step by step sequence of how this user story runs*

1. System shows a list of players in the game for the current_player to choose one
2. Player chooses the player whom the want to rob
3. System asks that player to choose a card to give you
4. if they have any cards, that player chooses a card from their hand
    a. System adds that card to this players hand
    b. System removes that card from the robbed player's hand

<u>Alternate Flows</u> *#anything that can deviate from the normal sequence*

- Can a player ever have 0 cards in their hand?

<u>Postconditions</u> *#what is guaranteed to be true after this use case runs.*

- Moves back to PlayATurn step 10

10. **<u>Attack</u>**

<u>Actor</u> *#who or what triggers this user story?*

- Player selected this card

<u>Preconditions</u> *#what must be true before this use case can run?*

- It is this players turn

<u>Main Flow</u> *#step by step sequence of how this user story runs*

1. System sets “is_attacking” to true
2. System sets turns_to_take of game state **+2**
3. System sets “skip_draw” field to True

<u>Alternate Flows</u> *#anything that can deviate from the normal sequence*

- Can a player ever have 0 cards in their hand?
- What if the other player takes too long to respond? Or how can we handle input from many people?

<u>Postconditions</u> *#what is guaranteed to be true after this use case runs.*

- Moves back to PlayATurn step 10

11. **<u>Nope</u>**

<u>Actor</u> *#who or what triggers this user story?*

- Any player except current player
    - We could have a field that says “is_current_player” or a method for game state to know who is the current player.

<u>Preconditions</u> *#what must be true before this use case can run?*

- The game is active (GameState status == "active")
- Another player just played an action card (it is NOT this player's turn)
- The action card is still in a "pending" state

<u>Main Flow</u> *#step by step sequence of how this user story runs*

1. Current player plays an action card
2. System puts that action into a "pending_state” on GameState (pending_action = that card)
3. System starts a Nope window timer (e.g. 5 seconds)
4. System notifies all other players "Player X played [CARD] — anyone want to Nope?"
5. If a player plays a Nope:
    a. System removes the Nope card from that player's hand
    b. System adds the Nope card to the DiscardPile
    c. System increments nope_count by 1 on GameState
    d. System resets the timer
    e. System notifies all players "Player Y played a Nope — anyone want to Nope the Nope?"
    f. Return to step 3 (reset timer and wait for another Nope or timer expiry)
6. When timer finally hits zero, system evaluates nope_count:
    a. If nope_count is odd (1, 3, 5...): result = false (action cannot be done)
    b. If nope_count is even (0, 2, 4...): result = true
7. Reset nope_count to 0
8. Return result (being returned at step

<u>Alternate Flows</u>

- Two players try to Nope simultaneously: System should queue them (process one nope then the next) — worth thinking about in a real time multiplayer context

<u>Postconditions</u>

- nope_count is reset to 0
- pending_action is cleared from GameState
- All Nope cards played are in the DiscardPile
- The original action either resolved or was cancelled depending on final nope_count parity
- Control is back in Play a Turn at the correct step

<u>Play a Turn doesn't need to know anything about timers, nope_count, or broadcasting. It just awaits a boolean and acts on it.</u>

12. **<u>Two Same Cards</u>**

<u>Actor</u> *#who or what triggers this user story?*

- Player selected these cards

<u>Preconditions</u> *#what must be true before this use case can run?*

- It is this players turn

<u>Main Flow</u> *#step by step sequence of how this user story runs*

1. System shows a list of players in the game for the current_player to choose one
2. Player chooses the player whom the want to rob
3. System asks player to choose a card from their deck (should be a blind selection so we can ask them to write any number within the length of the deck)
4. System adds that card at that index to this players hand
5. System removes that card from the robbed player's hand

<u>Alternate Flows</u> *#anything that can deviate from the normal sequence*

- Can a player ever have 0 cards in their hand?

<u>Postconditions</u> *#what is guaranteed to be true after this use case runs.*

- Moves back to PlayATurn step 10

13. **<u>Three Same Cat Cards</u>**

<u>Actor</u> *#who or what triggers this user story?*

- Player selected these cards

<u>Preconditions</u> *#what must be true before this use case can run?*

- It is this players turn

<u>Main Flow</u> *#step by step sequence of how this user story runs*

1. System displays a list of all other active players for the current player to choose from
2. Player chooses the player whom they want to rob
3. System prompts the player to name a specific card type they want
4. Player says what card they want.
5. System checks if that card exists in that player's hand.
6. If yes,
    a. System adds that card to this players hand
    b. System removes that card from the robbed player's hand
7. If no,
    a. System just displays "client doesn't have that card"

<u>Alternate Flows</u> *#anything that can deviate from the normal sequence*

- What if the user asks for an exploding kitten card? Impossible no one can have an exploding kitten card and they waste their selection

<u>Postconditions</u> *#what is guaranteed to be true after this use case runs.*

- Moves back to PlayATurn step 10